LLM-Assisted Quantum Circuit Optimisation: A Systematic Study

Snehal Raj

(Dated: Public snapshot: 26 October 2026.)

Quantum algorithms need arithmetic circuits with low resource cost and reliable coherent computation. Our pipeline combines an arithmetic edit grammar, optional large language model (LLM) guidance, and independent circuit audits. It produces 46 of 215 points on a frozen ECDSA.fail qubit–Toffoli Pareto frontier, a share of 21.4%. Our 52 submissions have distinct commits; 46 remain on the frontier, giving an 88.5% success rate among these commits. Our lowest qubit–Toffoli product submitted to the leaderboard is 1.008×10^9. This circuit has ninefold lower observed failure than the 24 October public comparator on matched inputs, at a 10.10% higher resource product. Controlled studies test which grammar components and model feedback improve the search. We also share the quantum-circuit-optimiser repository to reproduce the circuits and run model-free or LLM-assisted searches with independent checks.

I. INTRODUCTION

Quantum circuit optimization can act on gates and on the arithmetic programs that generate them. Arithmetic in Shor's algorithm implements classical functions coherently, which exposes Boolean logic as a useful design level [2]. A valid circuit must preserve the required phase and restore its temporary qubits. Its cost depends on which intermediate values the design stores, recomputes, or clears through measurement. Gidney's temporary logical-AND construction shows how measurement and phase correction reduce the cost of reversible arithmetic [3, 4]. Classical simulation with phase tracking makes repeated tests practical for circuits proposed by AI [4, 5].

ECDSA.fail benchmarks reversible secp256k1 point addition using public circuits and a shared evaluator [1]. The challenge follows Google's resource estimates and verification tools [2]. Its score multiplies qubit count Q by rounded mean executed Toffolis T. The Open Autoresearch paper reports an 86.1% product reduction by July 2026, reaching 1,151 qubits and 1,299,453 mean Toffolis [1]. The public record also contains a Q/T Pareto frontier. A circuit is Pareto-optimal if no other eligible circuit reduces one resource without increasing the other. Independent failure tests let us compare reliability alongside these resource costs.

We collect public circuit strategies and express them as a typed *edit grammar*: the arithmetic constructions available to the search. Each rule specifies an edit and the conditions under which it applies. Model-free evolution and LLM-assisted search use the same grammar and evaluator. Our contributions are:

- Our pipeline produces 46 of the 215 points on the frozen public Pareto frontier, a share of about 21.4%.
- Our lowest leaderboard submission has $QT = 1.008 \times 10^9$. This circuit has ninefold lower observed failure than the 24 October public comparator on matched inputs.

The library implements these edits and pairs model-free or LLM-assisted search with independent audits. Controlled studies test the effects of grammar choices and model feedback.

II. ECDSA.FAIL LEADERBOARD RESULTS

The qubit–Toffoli Pareto curve compares circuits across workspace budgets: each point offers a tradeoff between the two resources. Our pipeline contributes 46 of its 215 points, a share of 21.4% (Figure 1, Table I). These circuits span 797–1095 qubits and 1.818–14.390 million mean Toffolis. Forty-four points build on collaborators' packed Euclidean inversion circuits, nicknamed *Froghop*, which share storage between shrinking remainders and growing inverse coefficients. We extend this family through measurement-based cleanup, exact replacements of short gate blocks, and changes to workspace allocation.

Table II groups our measured submissions by arithmetic family and reports their largest resource reductions. Within packed inversion, extra carry storage saves up to 271,963 published mean Toffolis at a cost of 20 qubits. Cleanup and exact gate replacements also reduce Toffolis at fixed width; a paired 818-qubit test isolates a 20,206-Toffoli saving from gate replacements. The other two frontier points use circuits that store inverse coefficients in shared workspace, nicknamed *SKY-COF*. Gate replacements save Toffolis at fixed width, while changes to cofactor storage save up to six qubits with more arithmetic. Gate replacements give frontier points at 1,082 and 1,095 qubits.

Our lowest leaderboard submission has $QT = 1{,}008{,}123{,}460$, from 1,270 qubits and 793,798 mean Toffolis. It descends from a family that couples an inversion walk with arithmetic replay, nicknamed *Leapfrog*. We first make its approximation settings more conservative to reduce failures, then tune precision, traversal length, and workspace to recover resource cost. The selected circuit reduces QT by 8.99% from the repaired 1,292-qubit control (Table IV). On 32,768 matched inputs, the public parent, repaired control, and selected circuit fail 92, one, and nine times, respectively. The selected circuit therefore spends some of the reliability margin gained by the repair to reduce cost. Against the fixed 24 October

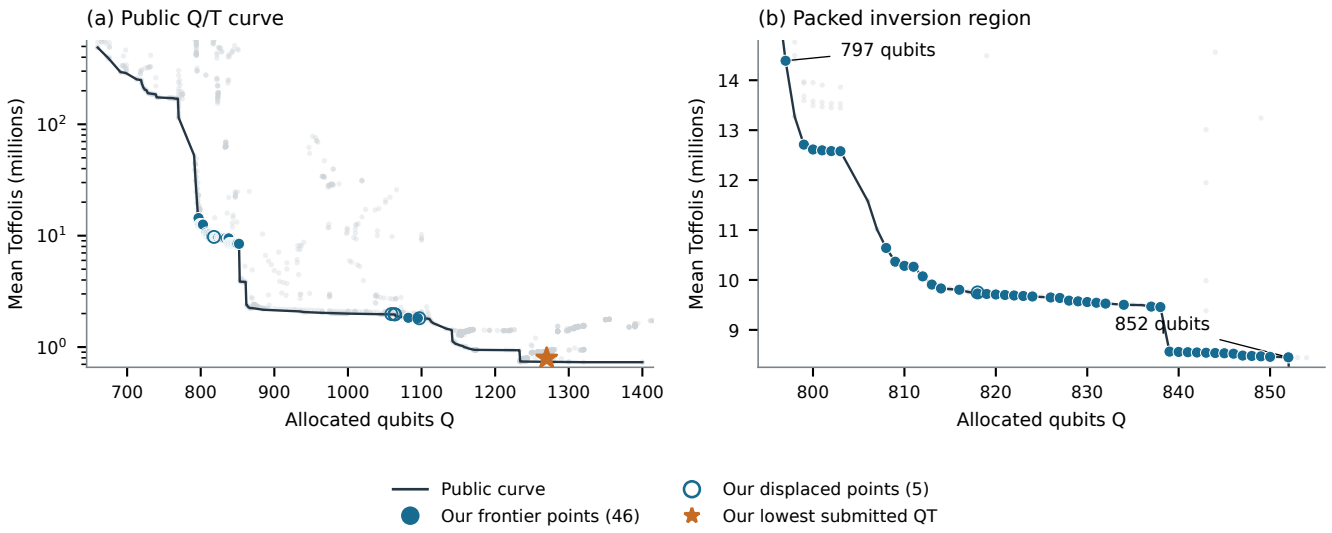


FIG. 1. **Our circuits on the ECDSA.fail frontier.** (a) The public Q/T frontier at 12:05 UTC on 26 October 2026. Blue points mark our 46 frontier coordinates at the cutoff; open points mark our five displaced coordinates. The star marks our lowest submitted QT, which lies outside the public frontier. (b) Detail of the packed-inversion region at 797–852 qubits.

TABLE I. **Frontier shares at the 26 October, 12:05 UTC snapshot.** Each coordinate belongs to its earliest published entry; frontier share divides by 215 coordinates. Successful commits gives the fraction of distinct submitted commits on this frontier. Our 52 submissions have distinct commits; 46 remain on the frontier (88.5%). Submissions include repeated commits and exclude unpublished trials.

User	Frontier share	Curve points	Submissions	Successful commits (%)	Q range on curve
odinfree	39.1%	84	312	27.9%	Q660–Q1400
gopikannappan	28.4%	61	232	26.5%	Q836–Q1320
SnehalRaj	**21.4%**	**46**	**52**	**88.5%**	**Q797–Q1095**
gnuchev	5.1%	11	167	6.7%	Q856–Q1164
jackylee0424	2.3%	5	129	4.4%	Q1158–Q1240
dysnasia	1.9%	4	46	8.7%	Q1170–Q1174
jieyilong	0.9%	2	103	2.2%	Q1133–Q1141
edi3on	0.5%	1	21	4.8%	Q1108
almond-lab	0.5%	1	8	12.5%	Q672

TABLE II. **Resource reductions across the three families we modify.** Rows order the largest Toffoli reductions from high to low. Each saving compares a submitted circuit with its declared parent; Leapfrog uses our repaired control. T differences use recorded rounded means on separate evaluation inputs. The two maxima can describe different circuits. The largest Froghop T saving adds 20 qubits; its largest Q saving adds 1,811,710 Toffolis. The largest SKY-COF Q saving adds 85,793 Toffolis. The Leapfrog circuit saves both 22 qubits and 63,584 Toffolis from the repaired control. Commits / curve gives measured commits and their points on the frozen frontier. All 52 submissions have published resource counts.

Family	Edits	Commits / curve	Q range	Max. T saving	Max. Q saving
Froghop	Cleanup, gate replacement, and workspace allocation	45 / 44	797–852	271,963	6
Leapfrog	Approximation tuning after reliability repair	1 / 0	1270	63,584	22
SKY-COF	Gate replacement and cofactor storage	6 / 2	1059–1097	1,119	6

public comparator, it has ninefold lower observed failure rate at a 10.10% higher resource product. A separate audit on 262,144 fresh inputs gives a failure-rate bound of 0.0406%, rounded upward. Section III D defines the observed failure rate and its statistical bound.

III. METHODS

A. Point-addition benchmark

ECDSA.fail tests point addition over the secp256k1 field, with modulus $p = 2^{256} - 2^{32} - 977$ [1]. The circuit adds a point given as classical input to a point held in a quantum register. The circuit must compute the sum, preserve the specified inputs, clear scratch qubits, and cancel unwanted phase. Reversible Boolean gates, phase operations, and measurement/reset allow efficient classical simulation of these checks. For mean executed Toffolis $\overline{T}$, the score is

$$T = \lfloor \overline{T} + 0.5 \rfloor, \qquad S = QT. \tag{1}$$

The mean depends on the evaluation inputs. The score excludes hardware routing, error correction, and measurement latency.

B. Public data and source selection

We collect 2,211 public submissions at the cutoff, including resource counts, research notes, source commits, and evaluation results. We use these records to identify the main frontier approaches (Table III, Figure 2). We use circuit implementations from the supplemental repository [6].

We fix the source versions before each experiment and record their settings and authors. The compiler study keeps inherited approximation settings fixed; the separate product search varies those settings.

C. Packed inversion and our edits

Froghop implements modular inversion through a reversible extended Euclidean algorithm [7]. Cofactors record the coefficients needed to recover the modular inverse. Each of two registers stores a remainder and a cofactor from opposite ends. Shrinking remainders leave space for growing cofactors, which reduces the storage needed for inversion. A shared boundary tracks their separation, while quotient bits control the cofactor updates. The one-hop version combines a cofactor update with alignment for the next division; the two-hop version performs these stages separately. Our 44 packed-inversion frontier points use both versions: six circuits at 797–803 qubits use two-hop updates; the remaining 38 use one-hop updates.

For a quantum point (x, y) and classical addend (x_0, y_0), the circuit first forms $d = x - x_0$ and $e = y - y_0$. Froghop inversion computes the slope $\lambda = e/d$. The coordinate products then give $d' = \lambda^2 - d - 3x_0$ and $n = -\lambda d'$. A second inversion recovers $\lambda = -n/d'$ to repair the phase after slope erasure. The output is $(x', y') = (d' + x_0, n - y_0)$, with all arithmetic modulo p and nonzero division denominators. Figure 3 shows these circuit blocks and an AND cleanup used by our edits.

Our cleanup edits enable `u` and `r` where the parent omits them. Option `u` measures an AND value and repairs its phase; option `r` absorbs unread writes before measurement/reset into phase operations. The inherited arithmetic, traversal schedules, approximation settings, and nonces stay fixed. We apply these passes to the emitted circuits, then run the public checker and fresh audits. The compiler-pass study below measures the isolated and combined savings.

The gate-replacement and layout edits use the 818- and 839-qubit one-hop constructions and the 803-qubit two-hop construction [8]. The catalog replaces short blocks on three or four wires with cheaper blocks that have the same signed permutation. It saves 20,206 Toffolis in the Q818 paired test. We vary scratch-pool sizes and carry-lane counts to obtain different qubit–Toffoli tradeoffs. The carry-lane edits preserve ordinary majority-gate lanes during the second inversion, as required by its rolling prefix and inverse repair. These edits preserve the inherited nonce and approximation settings.

D. Approximation tuning and evaluation inputs

An *approximation setting* limits arithmetic resources while permitting failure on rare inputs. Examples include a finite walk horizon and a narrowed comparison window. We tune these settings to balance resource cost against the *observed failure rate*, the fraction of independent inputs that fail any required check. Each input uses two measurement streams and counts as failed if either stream fails. We report a *failure-rate bound* to account for uncertainty from the finite sample. For $k < n$ failures in n inputs, the one-sided exact binomial bound is

$$U = \mathrm{Beta}^{-1}(1 - \alpha; k + 1, n - k). \tag{2}$$

Throughout this work, the joint-check audit limit is $U \leq 0.001$. Within each declared group of m planned audits, we allocate $\alpha = 0.05/m$ to give that group simultaneous 95% coverage. The packed-inversion audit records reserve $m = 32$ per group; the matched public panel uses $m = 16$, and the larger product confirmation uses $m = 2$. The fine-tuning pilot uses a separate provisional classical-only screen, described in Section IV A.

The public evaluator derives 9,024 shots from the serialized circuit hash [1]. A *nonce* changes that hash through redundant operations. Searching nonces can select a clean public test without changing the circuit's independent failure probability. Arithmetic edits also change the hash, so we audit selected circuits on fresh inputs (Table IV).

The packed-inversion edits preserve each parent's approximation settings and nonce. The lowest-QT circuit instead uses settings selected by our search. For arithmetic-variant screening, we run the public evaluator once per distinct variant. In the screening study frozen on 25 October, 19 of 31 shared-frame and Froghop tests pass and

TABLE III. **Arithmetic families in the public sources.** The public aliases identify the source lineages used in our analysis.

Family	Public alias	Main mechanism
Quotient recomputation	Frogdrop	Retain fewer quotient bits and regenerate omitted work; trade storage for more arithmetic.
Packed Euclidean walk	Froghop	Pack remainder and cofactor state; choose one-hop or two-hop updates and reuse the shared boundary.
Cross-packed divsteps	Lilypad	Use divsteps in packed rings; tune run storage and exploit known-zero cofactor bits.
Shared cofactor frame	SKY-COF	Carry cofactors in one frame; adjust rail widths and history allocation.
Recycled-history walk	Skywalk	Store decisions in wires released during arithmetic.
Coupled walk and replay	Leapfrog, Spookyfrog	Couple inversion progress and payload work; adjust fusion, precision, horizon, and workspace.
Recorded Euclidean replay	Dialog, Jump-2, Ping-Pong	Store a compact decision transcript and replay it; compress decisions or alternate registers.

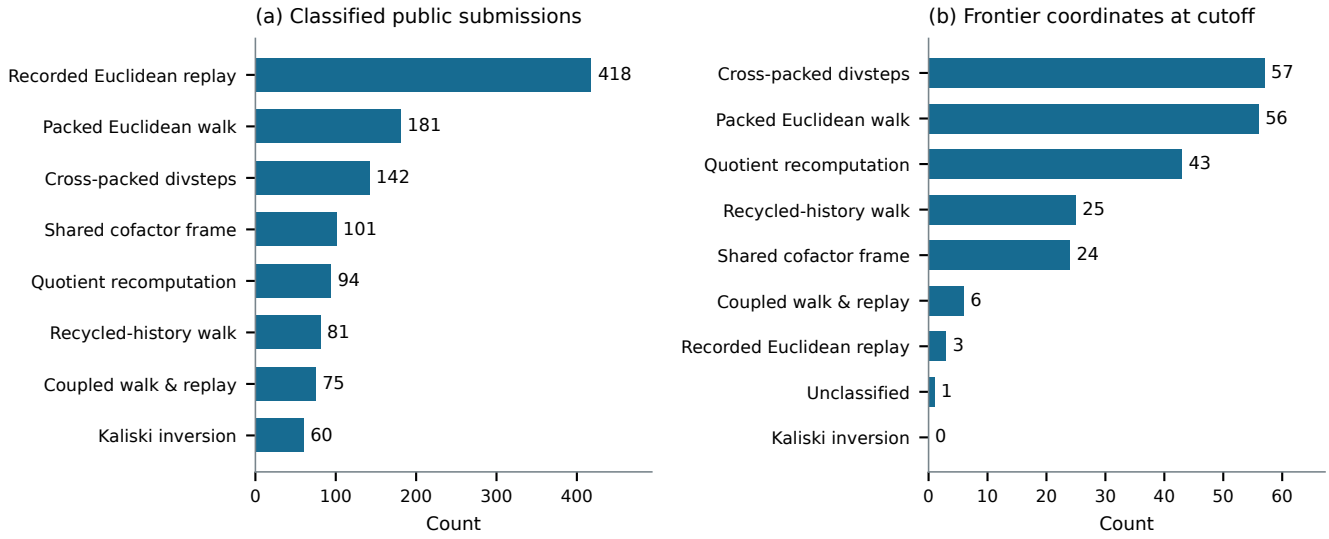


FIG. 2. **Circuit families named in public notes.** (a) The 1,152 submissions with identified families among 1,778 validated results, including repeated submissions. (b) All 215 frontier coordinates, including one with an unidentified family. Each panel orders families by decreasing count. Panel (a) omits 626 unclassified submissions. The labels come from submission notes.

TABLE IV. **Resource cost and observed failures.** The 1,292-qubit control uses more conservative arithmetic settings than the Leapfrog parent. Resource counts come from the recorded evaluation for each circuit. Failures use the same 32,768 inputs, with two measurement streams per input.

Circuit	Q	T	$QT/10^9$	Failures
24 Oct. comparator	1,234	742,031	0.9157	81
Leapfrog parent	1,236	768,542	0.9499	92
Control	1,292	857,382	1.1077	1
Our lowest QT	1,270	793,798	1.0081	9

12 fail. The parents contain nonce values selected by their authors. Our audits use independent inputs after candidate selection.

E. Search and submission pipeline

Figure 4 connects the setup, fast search loop, and submission phase. The following steps use the same edit interface with or without model feedback.

Build the benchmarks. The fast stage tests full 16-bit point addition with Kaliski inversion; the submission phase tests optimized secp256k1 circuits. Both evaluators check coordinates, phase, scratch, and preservation of the classical input point (Table V).

Define legal edits. Each grammar rule records its source, applicability conditions, implementation, and known counterexamples. Shared rewrites need implementations at both widths; layout settings without a 16-bit counterpart run directly at full width.

Select a parent and propose edits. Model-free evolu-

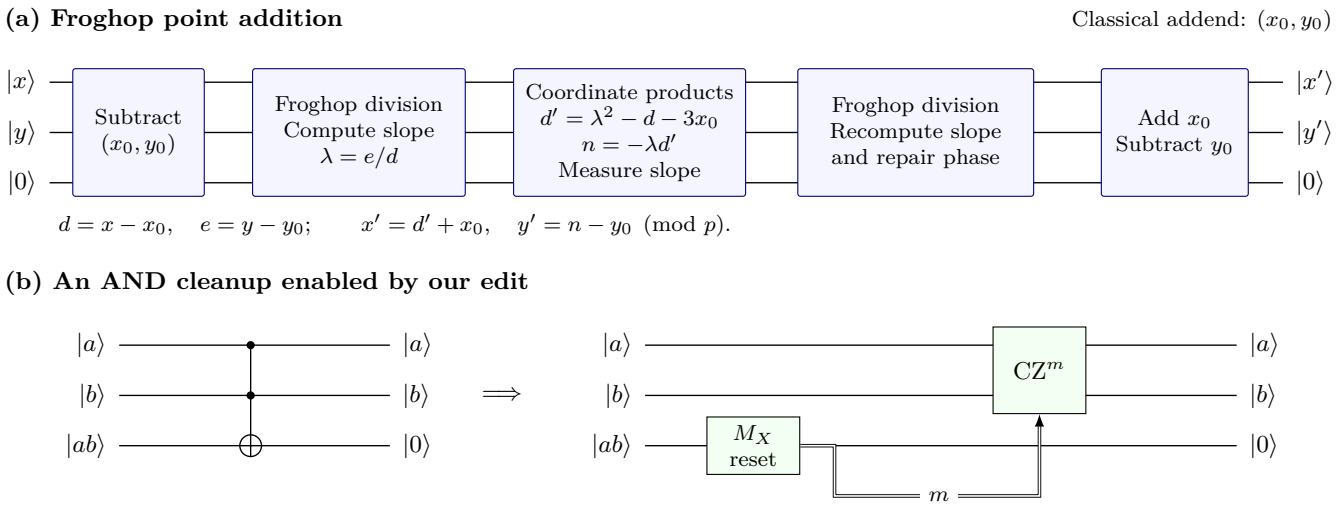


FIG. 3. **Froghop point addition and the cleanup used in our submissions.** (a) Circuit blocks follow the frozen source constructors [7]; the third wire groups the temporary qubits. The division blocks include forward and reverse Euclidean traversals. Measurement and later phase repair allow the circuit to reuse registers. (b) The simplest instance of option `u` replaces a Toffoli erasure of ab with X-basis measurement, reset, and CZ^m. The measurement result m controls the phase repair; both operand values must remain available [3, 4]. Our edits change the cleanup passes around the inherited arithmetic.

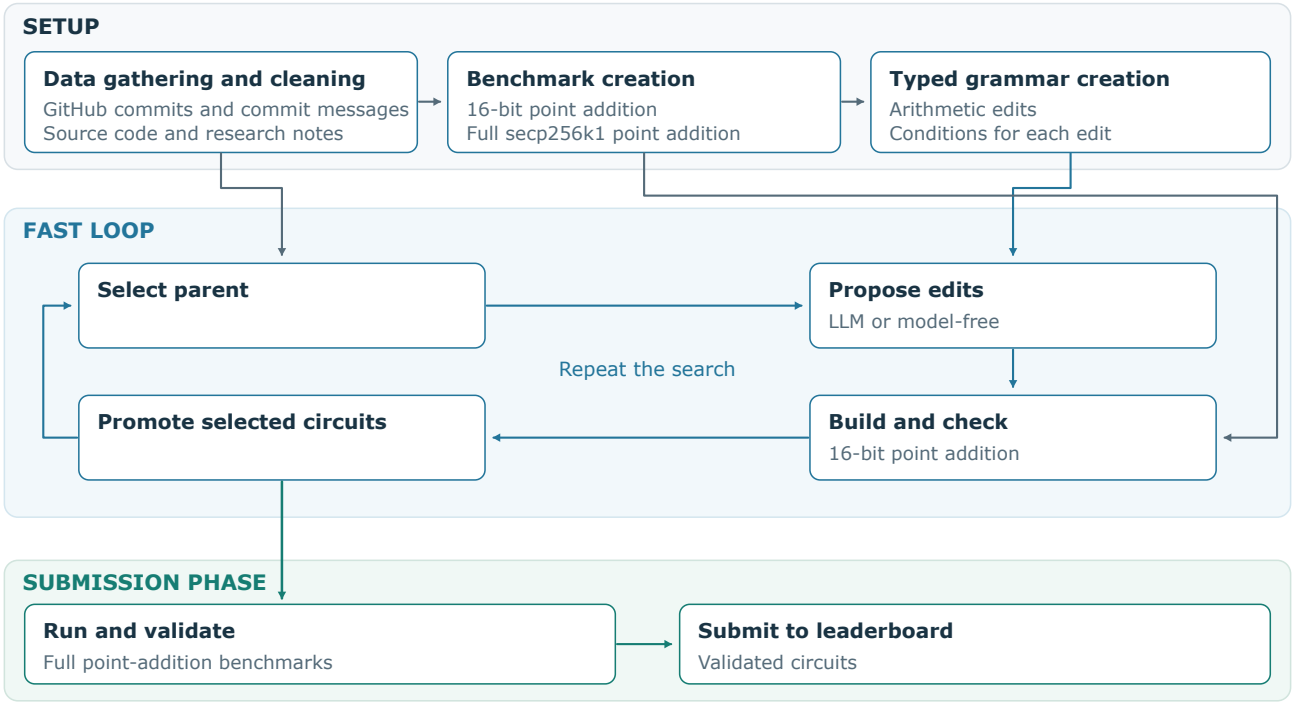


FIG. 4. **Design of the staged point-addition search.** The fast loop uses full 16-bit point addition; the submission phase uses optimized secp256k1 constructors. Model-free search and optional Claude or GPT feedback share the edit interface. Shared rewrites need an implementation at both stages. Full-width layout settings use the secp256k1 stage directly and record why they have no small-stage mapping. Selected circuits undergo reference checks and fresh audits before submission. Audit inputs remain separate from development inputs. The experiments evaluate the components separately. The model comparison uses the full 16-bit environment.

tion selects a parent that passes the development checks and applies a legal grammar edit. The model-assisted version also receives edit recommendations based on evaluation feedback; both versions retain valid circuits as possible parents.

Build and check. The fast evaluator checks each candidate's coordinates, phase, scratch, and classical input point. Invalid and duplicate proposals consume their search slots.

Retain and promote. Valid circuits remain available as parents, and selected Q/T tradeoffs advance to full-width evaluation. Each full-width candidate needs its own checks and fresh audit, including edits that have no small-stage implementation.

Audit before submission. We freeze each selected circuit before testing fresh inputs under the joint-check protocol in Section III D.

Submit and record the result. We check the live curve before submitting a qualifying candidate from its frozen source checkout. The submission record links the parent, edits, audits, and leaderboard result.

Grammar study. We enumerate the 16 combinations of two cleanup options, cofactor-register release, and a combined coordinate construction on one 16-bit Kaliski parent. Nested grammars separate the cost reduction available from each edit class (Table VI, Figure 5a). Cleanup reduces QT by 11.70%; adding register release raises that reduction to 18.60%.

Model comparison. Random search, evolution, Claude, and GPT each receive eight proposals from the same grammar, parent, and paired input sets (Table VII). Models see rule descriptions and past evaluations; they receive no results from the exhaustive grammar study. Claude's first proposal reduces QT by 17.57% at the median, compared with 7.77% for evolution. By eight proposals, evolution and both models share the same 18.60% median reduction.

Compiler study. We apply gate replacements, Boolean factoring, and algebraic cancellation to 36 public sources (Figure 6). Paired checks compare parent and child; measured cleanup must also repair phase while the required operands remain available [3, 4].

IV. LIMITATIONS

We now document what fell short in fine-tuning and what prevented us from extending the search to new arithmetic families. The fine-tuning study found transferable setting changes but produced few usable edits. Our executable grammar likewise supports local improvements, while broader arithmetic rules still require implementations and checks. These outcomes suggest separate next steps for reliable proposal generation and for a larger space of circuit designs.

A. Fine-tuning study

We fine-tune Qwen2.5-Coder-7B-Instruct on recorded circuit edits to test whether it can propose useful changes on held-out parents [9]. The study compares supervised training, feedback during search, and further training on model-generated examples (Table VIII). We also train on frontier examples that pair source changes with resource measurements, using a control with the same adapter rank. These experiments ask whether examples of successful edits teach the model which arithmetic settings affect cost.

Two obstacles limit the resulting proposals. First, settings in the source can leave the selected constructor unchanged. A probe finds that only 20 of 206 tested settings change the circuit. Second, many generated edits either fail to compile or leave the emitted circuit unchanged. Only 7.8% of outputs in the self-generated training study both compile and change the circuit. This leaves too few useful examples for the planned training round. Adding evaluation feedback also gives zero accepted edits in 48 attempts, compared with one in 48 without feedback.

The setting choices themselves show more promise. Applying model-discovered changes through a script yields 34 provisionally accepted edits across 13 held-out parents; random setting moves yield zero in 111 attempts. This suggests separating the choice of an arithmetic setting from the code change that implements it. A model could select a setting and direction, while a checked transformation applies the edit and records its effect on the circuit. Figure 7 places this proposal step within a training and evaluation loop.

The strongest retained control candidate reduces QT by 2.12×10^6 and has 108 classical failures in 72,192 inputs, an observed rate of 0.1496%. The pilot accepts a classical failure rate up to 0.15%; phase and scratch failures are outside that provisional screen. Its accepted edits also shorten an inherited traversal horizon, which can exchange reliability for cost. We therefore leave independent joint-check audits and paired ablations of the setting changes and traversal length for future work. The frontier-example arm reaches one parent with provisional improvements, compared with three for the rank-matched control in the incomplete evaluation. A larger held-out study can test whether the proposed separation improves edit yield and whether frontier examples add useful information.

B. Search over arithmetic programs

Our 46 frontier points come from deterministic compiler and layout edits within two existing inversion families. These edits expose savings in cleanup and storage, while retaining the parent's arithmetic program. Larger changes require a search space that can alter how the circuit computes and retains intermediate values. We explored this direction by recording 23 broader arithmetic rules; turning those descriptions into executable transformations

TABLE V. **Validation of full point addition on 100,000 input pairs per circuit.** Each pair uses two measurement streams. Time covers validation and excludes circuit generation and compilation. The runs use an Apple M2 workstation with eight cores and 24 GiB of memory. The 16-bit circuit uses Kaliski inversion with policy `mfsur`; the secp256k1 circuit is our lowest-QT Leapfrog circuit. Mean T uses the timing inputs.

Environment	Q	Mean T	Evaluator	Threads	Time
16-bit point addition	214	28,051	Rust	1	35.15 s
secp256k1 point addition	1,270	793,801	Rust	4	144.32 s

TABLE VI. **Cost reachable with each point-addition grammar.** All rows start from the same parent after the common `mfs` compiler preparation. G0 enables cleanup options `u` and `r`; G1 adds cofactor-register release; G2 also permits combined coordinate products. We enumerate every combination and report each grammar's lowest $Q\overline{T}$ among circuits that pass the joint-check audit. Gain measures the reduction from the prepared parent, excluding common preparation. Common preparation reduces the original parent's cost by 5.81%. All costs use the same fresh audit inputs and mean executed Toffolis before rounding.

Grammar	Available edits	Circuits	Q	Mean T	$QT/10^6$	Gain
Parent	Common preparation	1	214	31,769.0	6.799	0.00%
G0	Cleanup	4	214	28,051.0	6.003	11.70%
G1	+ register release	8	198	27,949.0	5.534	18.60%
G2	+ combined products	16	198	27,949.0	5.534	18.60%

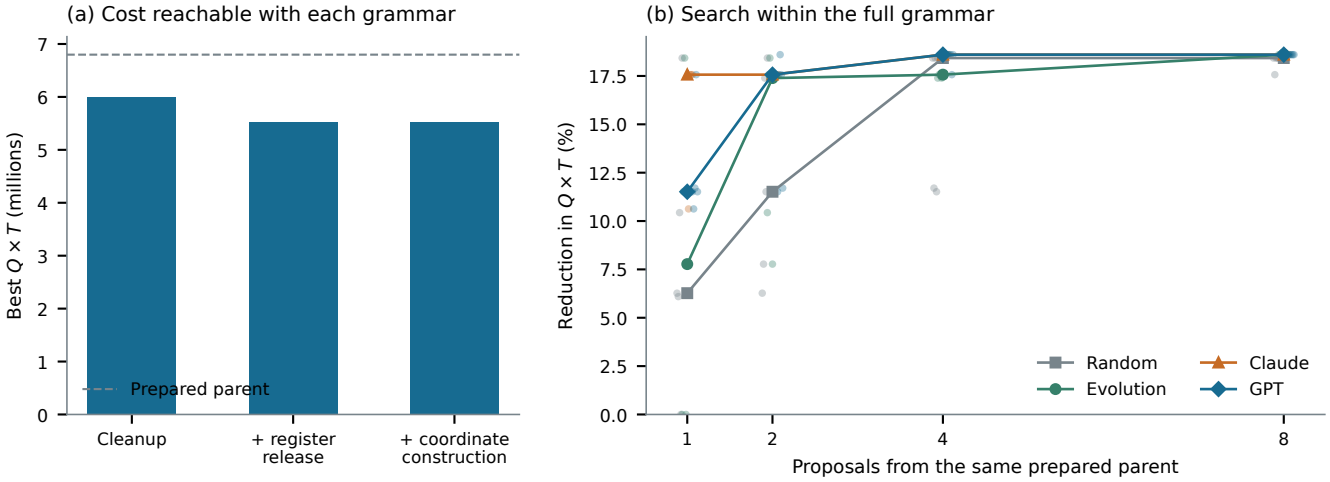


FIG. 5. **Grammar capacity and search efficiency on full 16-bit point addition.** (a) The lowest resource cost that passes the joint-check audit in each nested grammar. (b) Best cost reduction after 1, 2, 4, and 8 proposals within the full grammar. Lines show medians over five runs; faint points show individual runs. Every run starts from the same prepared parent. Each method chooses two proposals before receiving their evaluation results.

TABLE VII. **Search from one full 16-bit point-addition parent.** Values are median reductions in $Q\overline{T}$ across five paired runs, measured from the prepared parent. Best by 4 or 8 counts runs that find the grammar's minimum within that budget. Evolution favors one-setting edits; random search samples all unseen recipes uniformly. Models choose from the same unseen recipes, with feedback after each pair. Failed calls consume slots; Calls reports completed/attempted counts. The CLIs retain their default contexts; GPT's resolved model ID is unreported. The five runs vary input packets and menu order; the subscription CLIs do not expose a sampling seed. All 17 circuits have zero failures on 16,384 fresh inputs after finalists freeze. At $\alpha = 0.05/20$, each failure-rate bound is 0.0366%, rounded upward.

Proposer	1 proposal	2 proposals	4 proposals	8 proposals	Best by 4	Best by 8	Calls
Random	6.27%	11.51%	18.43%	18.43%	0/5	0/5	0/0
Evolution	7.77%	17.39%	17.57%	18.60%	0/5	4/5	0/0
Claude Opus 5.5	17.57%	17.57%	18.60%	18.60%	5/5	5/5	20/20
GPT 6.1 Sol	11.51%	17.57%	18.60%	18.60%	4/5	5/5	20/20

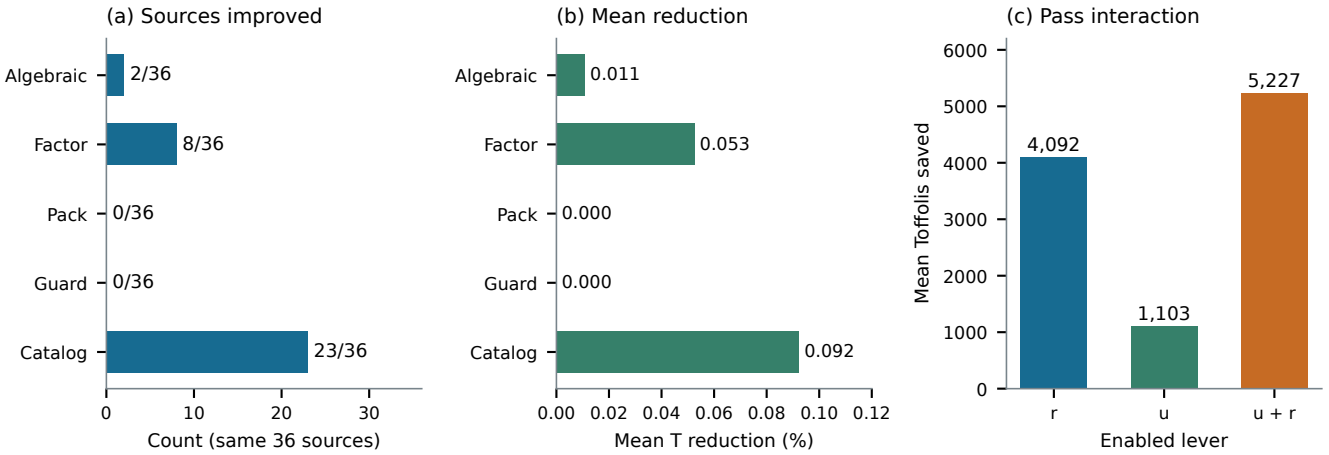


FIG. 6. **Compiler-pass opportunities in public circuits.** (a) Sources with lower T after each pass. (b) Mean percentage reduction across all 36 sources, including unchanged circuits. (c) Isolated and combined cleanup options on the same 839-qubit parent. All comparisons use the same 8,192-input development protocol.

TABLE VIII. **Fine-tuning experiments and their outcomes.** Accepted edits use the provisional classical-only screen described in the text. The frontier-example comparison is incomplete. The public frontier results use deterministic compiler and layout edits.

Experiment	Observed result
Supervised adapter training	No generated candidate improves its parent's measured score in the initial evaluations.
Feedback during search	0/48 accepted edits with feedback; 1/48 without feedback.
Scripted transfer of setting choices	34/88 accepted edits on 13 parents; random setting moves give 0/111.
Training on model-generated examples	7.8% compile and change the circuit, below the 10% study target.
Deterministic repair of failed edits	2/64 edits compile and change the circuit after repair, below the target of five.
Frontier examples and rank-16 control	Provisional improvements on one parent versus three for the control.

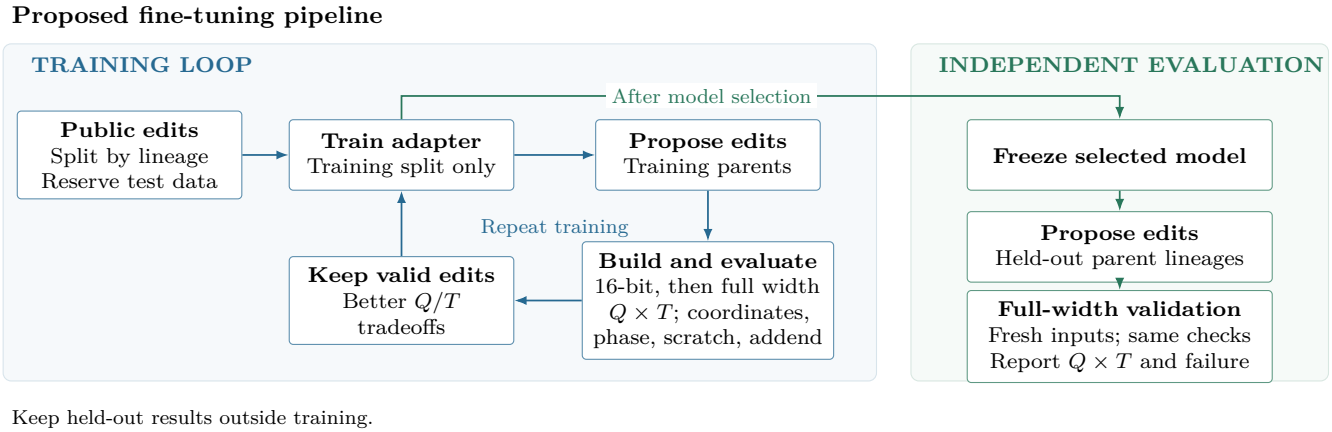


FIG. 7. **Proposed fine-tuning workflow.** The training loop reuses valid improvements. Independent evaluation uses a frozen model, held-out lineages, and fresh inputs. The retained study tested parts of this design with classical-only acceptance. Small-stage tests apply where the edit has a 16-bit implementation.

remains unfinished work.

The controlled model study motivates testing a larger edit space. It compares 16 edit combinations on one 16-bit Kaliski constructor. Model guidance finds good combinations earlier, but evolution reaches the same median gain within eight proposals. Future comparisons should vary constructors and search budgets, with controls that match the number of edits allowed per proposal. They can then measure whether the early search gains persist as the grammar grows and at full field width.

We propose a grammar of *typed arithmetic programs* with explicit register roles and block requirements. Register roles would distinguish field values, carries, cofactors, and retained arithmetic decisions. The search could then choose between storing an intermediate value, recomputing it, or erasing it through measurement and phase correction. Measured cleanup makes one dependency especially important: the values needed for later phase correction must remain available. Tracking this dependency would let the search change storage schedules while preserving the conditions of each cleanup rule.

A first implementation could vary carry storage and cleanup before composing alternative inversion blocks with explicit cofactor tracking. Each transformation would need an equivalence argument and paired tests of coordinates, phase, scratch, and addend preservation. Full 16-bit point addition provides a tractable test environment before full-width evaluation. We leave the discovery of new inversion families to this broader search.

V. CONCLUSION

Our pipeline contributes 46 of 215 public frontier points through cleanup, exact gate replacements, and workspace changes in packed-inversion and shared-cofactor circuits. Approximation tuning produces a separate circuit with $QT \simeq 1.008 \times 10^9$ and ninefold lower observed failure rate than its fixed 24 October comparator. The controlled search study shows an early benefit from model guidance within a small edit space, while fine-tuning exposes difficulties in generating usable code changes. Together, these results motivate checked arithmetic transformations that let models propose changes and let evaluators measure their cost and reliability. The library packages the edits, search methods, and audit records needed to study that approach.

AI DISCLOSURE

The author used GPT-6 to help write this manuscript. The author used Claude Opus and GPT-6 to help develop the research harness.

CODE AND DATA

The quantum-circuit-optimiser repository packages the methods used for the submissions, together with the evaluation harness and experiment records. The model comparisons use full 16-bit point addition. The bundle contains source identities, frozen public records, study contracts, outcomes, and vector figures for an offline paper build. Public constructors retain their authors' licenses and notices. The lowest-QT circuit descends from krishparadox's Leapfrog submission `54acebaf`. The fixed 24 October comparator is Champollion9012's circuit at commit `6b0327d6`. The Froghop parents come from gnuchev and odinfree; jackylee0424 wrote the conditional-AND passes. The 1,097-qubit SKY-COF derivative builds on gopikannappan's `f22e87ab` and its credited lineage. The 1,082- and 1,095-qubit derivatives use odinfree's `d0abb28f` and `8a57279a`, respectively. The carry-lane derivatives follow gopikannappan's `e2194147` setting and guard.

[1] J. Long *et al.*, *ECDSA.Fail: Open Autoresearch for Optimizing Elliptic-Curve Point Addition in Shor's Algorithm*, arXiv:2609.09582v2 (2026).

[2] R. Babbush *et al.*, *Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations*, arXiv:2603.28846v2 (2026).

[3] C. Gidney, *Halving the cost of quantum addition*, Quantum **2**, 74 (2018), doi:10.22331/q-2018-06-18-74.

[4] C. Gidney, *Verifying Measurement Based Uncomputation*, Algorithmic Assertions (7 August 2019), algassert.com/post/1903.

[5] T. Khattar, N. C. Rubin, C. Gidney, M. Broughton, G. H. Low, and R. Babbush, *Leveraging verifiable structure in quantum circuits to find more efficient compilations with agentic AI*, arXiv:2610.23832v2 (2026).

[6] J. Long and contributors, *ECDSA.fail supplemental materials*, source repository (accessed 25 October 2026).

[7] gnuchev and odinfree, *Froghop source constructors and submission notes*, ECDSA.fail (2026), commits `410bec6d` and `92186c37`. Q839 attribution; Q799 attribution. The bundle contains the frozen source files and claim map.

[8] SnehalRaj and credited contributors, *Froghop catalog and layout submissions*, ECDSA.fail (2026). Q818 catalog; Q828 carry-lane derivative. The frozen snapshot retains the submission notes and their source credits.

[9] S. Raj, *Circuit candidate-generator fine-tuning study*, commit `0f762c895` (2026). Retained experiment report. The bundle retains the records and recount.